

Huffman Coding — Module 7 Lab

Course: CSC 821 — Design and Analysis of Algorithms **Group:** Group 1 **Task:** Write a Python program that implements Huffman coding for a given set of characters and their frequencies.

Objective

Huffman coding is a *lossless* data-compression method. It assigns **short binary codes to frequent characters and long codes to rare ones**, so the total number of bits needed to store a message drops — without losing any information (we can always decode back to the exact original).

It is a classic **greedy algorithm**: at every step it makes the locally optimal choice (merge the two least-frequent items) and never reconsiders — and for this problem that greedy choice is provably optimal.

1. The greedy idea

Two rules drive everything:

1. **Frequent** → **short code**, **rare** → **long code**. That is where the saving comes from.
2. **To achieve rule 1, repeatedly merge the two *smallest* frequencies.** The rarest characters get merged first, so they sink to the *bottom* of the tree (long codes); the most frequent character is merged last, so it stays near the *top* (short code).

We use a **min-heap** (priority queue) so that "give me the two smallest" is fast — $O(\log n)$ per pop.

```
In [1]: import heapq
```

build_tree — the greedy merging

- Start with one heap entry per character: `(frequency, tie-breaker, node)`.
- A **node** is either a single character (a *leaf*) or a `(left, right)` pair (an *internal* node).
- Pop the two smallest, fuse them into a new node whose frequency is their **sum**, push it back.
- Repeat until a single node is left — that is the **root** of the Huffman tree.

The `counter` is only there to break ties deterministically when two frequencies are equal (so Python never has to compare two `(left, right)` tuples).

```
In [2]: def build_tree(frequencies):
        """Merge the two smallest frequencies until one node (the root) is left."""
        heap = [(freq, i, char) for i, (char, freq) in enumerate(frequencies.items())]
        heapq.heapify(heap)
        counter = len(heap)

        while len(heap) > 1:
            freq_a, _, node_a = heapq.heappop(heap) # smallest
            freq_b, _, node_b = heapq.heappop(heap) # second smallest
            heapq.heappush(heap, (freq_a + freq_b, counter, (node_a, node_b)))
            counter += 1

        return heap[0][2] # the root node
```

build_codes — read the codes off the tree

Walk down from the root. **Every left turn appends 0, every right turn appends 1.** When we reach a leaf (a `str`), the string we have accumulated is that character's code.

Because characters live only at the leaves, no code is ever a *prefix* of another — this is a **prefix code**, which is exactly what lets the decoder work with no separators between codes.

```
In [3]: def build_codes(node, prefix=""):
        """Walk the tree: left adds '0', right adds '1'. Leaves hold the final code."""
        if isinstance(node, str):
            return {node: prefix or "0"} # a leaf -> code is complete
            # 'or "0"' handles a 1-character a

        left, right = node
        codes = build_codes(left, prefix + "0")
        codes.update(build_codes(right, prefix + "1"))
        return codes
```

encode and decode

- **Encode:** look up each character's code and glue them together into one bitstring.
- **Decode:** walk the bits down the tree (`0` = left, `1` = right); each time we land on a leaf, emit that character and **jump back to the root**. No delimiters are needed — the prefix-code property guarantees the decoder always knows where one code ends and the next begins.

```
In [4]: def encode(text, codes):
        """Replace each character with its binary code."""
        return "".join(codes[char] for char in text)

def decode(bits, root):
```

```

"""Follow the bits down the tree, emitting a character at each leaf."""
text, node = [], root
for bit in bits:
    node = node[0] if bit == "0" else node[1]
    if isinstance(node, str):
        text.append(node)
        node = root
    # reached a leaf
    # start again from the top
return "".join(text)

```

A small driver

`run` ties it together: build the tree, read off the codes, print the code table, then encode and decode a message (and confirm the round-trip is lossless).

```

In [5]: def run(frequencies, text):
        """Build codes for these frequencies, then encode and decode text."""
        root = build_tree(frequencies)
        codes = build_codes(root)

        print(f"{'char':<6}{'freq':>6}{'code':>8}")
        for char in sorted(codes, key=lambda c: frequencies[c]):
            shown = repr(char) if char == ' ' else char
            print(f"{shown:<6}{frequencies[char]:>6}{codes[char]:>8}")

        encoded = encode(text, codes)
        print(f"\ntext      : {text}")
        print(f"encoded : {encoded} ({len(encoded)} bits)")
        print(f"decoded : {decode(encoded, root)}")
        print(f"lossless: {decode(encoded, root) == text}")
        return codes

```

2. Worked example — the module's frequency table

Characters `A-F` with frequencies `5, 9, 12, 13, 16, 45`.

```

In [6]: codes = run({'A': 5, 'B': 9, 'C': 12, 'D': 13, 'E': 16, 'F': 45}, "ABCDEF")

```

char	freq	code
A	5	1100
B	9	1101
C	12	100
D	13	101
E	16	111
F	45	0

```

text      : ABCDEF
encoded : 110011011001011110 (18 bits)
decoded : ABCDEF
lossless: True

```

Why these codes? Tracing the greedy merges:

Step	Two smallest merged	New node
1	A(5) + B(9)	AB = 14
2	C(12) + D(13)	CD = 25
3	AB(14) + E(16)	ABE = 30
4	CD(25) + ABE(30)	CDABE = 55
5	F(45) + CDABE(55)	root = 100

F (the most frequent) is only merged at the very last step, so it sits right under the root and gets the 1-bit code **0**. **A** (the rarest) was merged first, so it is buried deepest → **1100**. That is the greedy rule paying off.

3. Frequencies counted from real text

Here we build the frequency table straight from a string, so the code works on any input.

```
In [7]: message = "haraka haraka haina baraka"
counts = {}
for char in message:
    counts[char] = counts.get(char, 0) + 1

run(counts, message)
```

char	freq	code
b	1	1100
i	1	11010
n	1	11011
k	3	100
' '	3	101
h	3	1110
r	3	1111
a	11	0

```
text      : haraka haraka haina baraka
encoded   : 1110011110100010111100111101000101111001101011011010111000111101000 (67 b
its)
decoded   : haraka haraka haina baraka
lossless: True
```

```
Out[7]: {'a': '0',
         'k': '100',
         ' ': '101',
         'b': '1100',
         'i': '11010',
         'n': '11011',
         'h': '1110',
         'r': '1111'}
```

The letter **a** appears 11 times and earns the 1-bit code **0**; the rare **i** and **n** (once each) get 5-bit codes. The whole 26-character message packs into far fewer bits than fixed-width

ASCII — and decodes back exactly.

4. Compression analysis

How much do we actually save? Compare Huffman's **average code length** against a fixed-width code (which needs $\lceil \log_2(\text{number of symbols}) \rceil$ bits per character).

```
In [8]: import math

def analyse(frequencies):
    codes = build_codes(build_tree(frequencies))
    total = sum(frequencies.values())
    avg = sum(frequencies[c] * len(codes[c]) for c in frequencies) / total
    fixed = max(1, math.ceil(math.log2(len(frequencies))))
    print(f"symbols           : {len(frequencies)}")
    print(f"Huffman avg length : {avg:.3f} bits/char")
    print(f"fixed-width length : {fixed} bits/char")
    print(f"space saving          : {(1 - avg/fixed) * 100:.1f}%")

analyse({'A': 5, 'B': 9, 'C': 12, 'D': 13, 'E': 16, 'F': 45})

symbols           : 6
Huffman avg length : 2.240 bits/char
fixed-width length : 3 bits/char
space saving          : 25.3%
```

The skew in the frequencies is what Huffman exploits: the more uneven the character counts, the bigger the saving. On real text (where a few letters dominate) the gain is larger still.

5. Edge case — a single distinct character

If the alphabet has only one symbol there is no second node to merge, so the tree is a lone leaf. The `prefix` or `"0"` guard gives that symbol the code `"0"` instead of an empty string, so encoding and decoding still work.

```
In [9]: run({'Z': 7}, "ZZZ")

char    freq    code
Z        7        0

text     : ZZZ
encoded  : 000 (3 bits)
decoded  : ZZZ
lossless: True

Out[9]: {'Z': '0'}
```

Conclusion

- **Correctness:** every example round-trips (`decoded == text`) — the compression is lossless.
- **Why greedy works here:** always merging the two least-frequent nodes provably minimises the expected code length (the exchange argument: swapping any two codes toward this order never helps).
- **Complexity:** building the tree does $n - 1$ merges, each an $O(\log n)$ heap operation, so $O(n \log n)$ time; encoding and decoding are linear in the length of the message.

Huffman coding is the classic illustration that a strictly local, never-look-back rule can produce a globally optimal result — the defining property of a good greedy algorithm.